ADR-2026-39

No DAG Scheduler for ETL Job Queue

Status: Accepted | **Date:** 2026-03-11 | **Owner:** Daniel Shanklin / Director of AI & Technology

Related: ADR-2026-33 (SKIP LOCKED Job Queue), ADR-2026-26 (No Separate Post-Ingestion Worker)

BLUF (Bottom Line Up Front)

We will not build a DAG scheduler for the ETL job queue. Bronze extraction is inherently flat — objects then associations, two levels — and cross-source dependencies live at the silver/gold layer where materialized view refresh order handles them. Priority-based ordering plus a single _depends_on field is the correct tool at this scale and topology.

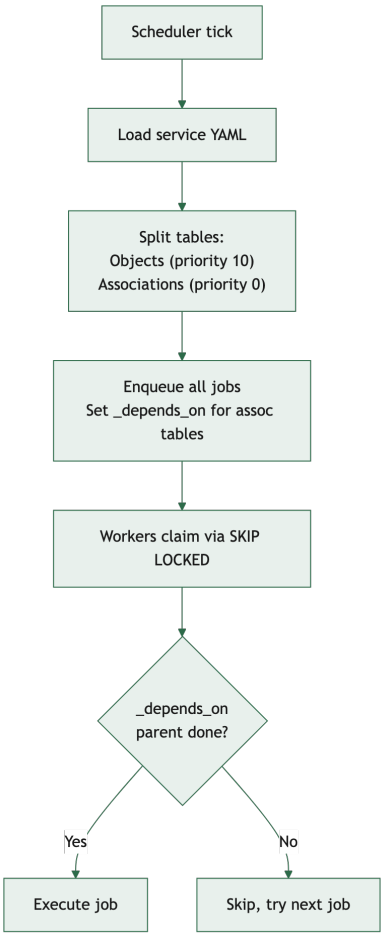
> **DECISION: Do not build a DAG scheduler. Keep priority-based ordering with single _depends_on dependency enforcement. The extraction topology is flat — each source is an independent two-level tree (objects → associations). Cross-source dependencies are handled by silver/gold view refresh, not the job queue.**

Context

During architecture review of the data-daemon ETL pipeline, we implemented job dependency enforcement: association tables (priority 0) are blocked until their parent object tables (priority 10) complete. This uses a _depends_on field in the job payload, checked by the claim_job SQL query via a NOT EXISTS subquery.

The question arose: should we build a proper DAG scheduler? The pipeline is growing from 1 data source (HubSpot, 33 tables) toward 10+ sources (Sage, Navusoft, fleet telematics, QuickBooks, ServiceTitan). We challenged the assumption that more sources requires a DAG.

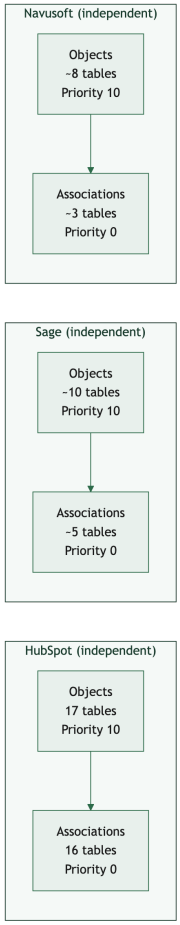
Current Architecture



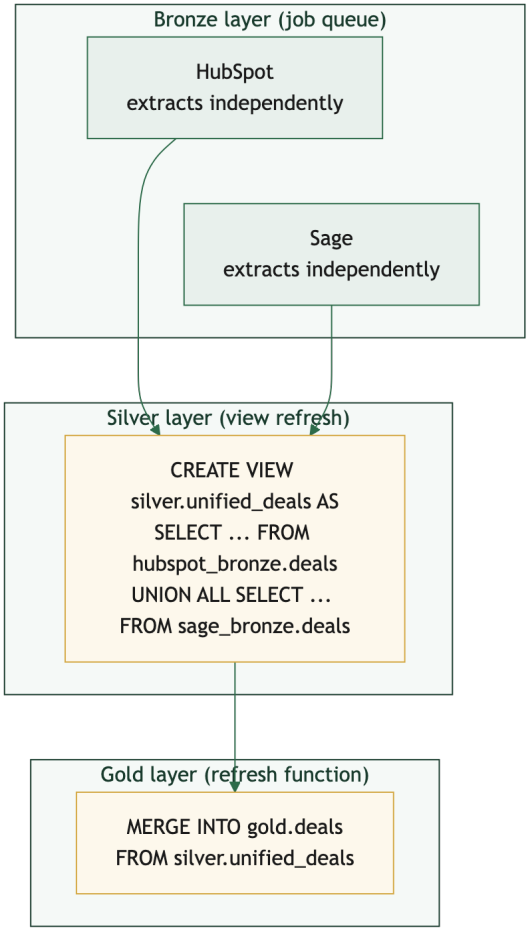
This handles the only dependency pattern that exists in bronze extraction: objects → associations. Two levels. No diamonds, no chains, no conditionals.

The Key Insight: Sources Are Independent

10+ sources sounds like it demands a DAG. But each source extracts independently — HubSpot doesn't wait for Salesforce, QuickBooks doesn't need Google Contacts. 10 independent two-level trees is not a DAG. It's 10 separate lists running in the same job queue.



Where Cross-Source Dependencies Actually Live



Cross-source joins happen at silver/gold via SQL view definitions, not at the extraction layer. The executor already refreshes views after loading their source tables. View refresh ordering is a simpler problem than DAG scheduling — and it's already solved.

Rationale

Factor	Assessment
Extraction is flat	Every source follows the same two-level pattern: objects → associations. No three-level chains, no diamonds, no conditional branches.
Sources are independent	10 sources = 10 independent two-level trees, not one 10-source DAG. SKIP LOCKED handles concurrency.
Cross-source = silver/gold	A silver view joining HubSpot + Sage is a REFRESH MATERIALIZED VIEW — view refresh ordering, not job queue ordering.
_depends_on handles it	Single string covers every real case: assoc_deal_contact depends on deals, assoc_call_deal depends on calls. No multi-parent bronze dependencies exist.
YAGNI	TopologicalSorter, cycle detection, multi-parent arrays solve problems we don't have. Maintaining unused code is negative value.
The YAML is the DAG	assoc_from_table: hubspot_bronze.deals is a human-readable dependency declaration. A runtime graph would produce identical ordering.

Options Considered

Option	Verdict
Keep priority + _depends_on (chosen)	Correct tool for flat topology. Zero new code. Already working.
Build DAG with graphlib.TopologicalSorter	Solves problems we don't have. ~100 lines with no current consumer.

Option	Verdict
Adopt Airflow / Dagster / Prefect	Massive infrastructure for 33 tables every 15 minutes. 1000x overkill.
Extend _depends_on to a list now	Pre-optimizing for multi-parent dependencies that don't exist in bronze.

Consequences

Positive: No new code to write, test, or maintain. Architecture stays simple. Mental model stays simple: 'objects first, associations second.' New data sources plug in with zero scheduler changes — just add a YAML file.

Negative: If a bronze table ever needs to depend on two different parent tables within the same service, the single _depends_on string won't handle it. Known limitation.

Mitigation: Extending _depends_on from a string to a list is a 30-minute change. The SQL subquery changes from checking one table name to checking ANY() on an array. Trivial upgrade, not a rewrite.

When to Revisit

• A bronze table needs two different parent tables within the same extraction service
• The executor needs to orchestrate cross-service extraction order
• Pipeline grows beyond ~500 tables and parallelism optimization matters
• Team grows beyond 3 people and 'just read the YAML' stops being sufficient